YENEPOYA UNIVERSITY

(Deemed to be University)

FINAL PROJECT REPORT

on

Cloud Cost Intelligence Platform for Smart City

Student Name:	Shuhaib Abdulla
Registration No:	23BCAICD101
Campus ID:	25296
Project Code:	PRJN26-108
Programme:	BCA – AI, Cloud Computing & DevOps
Industry Mentor:	Sumit Kumar Shukla
Guided By:	Ms. Deeksha Shetty
Academic Year:	2023–26

Executive Summary

The Cloud Cost Intelligence Platform for Smart City is a Python-based data analytics application developed to address the growing need for transparent and efficient cloud expenditure management in municipal infrastructure. As smart cities expand their reliance on cloud services spanning Compute, Storage, Networking, Database, and AI/ML capabilities, the ability to monitor, analyse, and optimise spending across multiple departments becomes a critical operational imperative.

Background

Modern smart city ecosystems generate substantial cloud infrastructure costs across diverse operational departments — Traffic Management, Public Safety, Smart Lighting, Waste Management, and Water Supply. Without a centralised analytics layer, municipalities face budget overruns, redundant provisioning, and an inability to make data-driven infrastructure decisions. This project addresses that gap by building a self-contained analytical pipeline that transforms raw cloud billing records into actionable visualisations and structured reports.

Aim

The aim of this project is to design, implement, and validate a Cloud Cost Intelligence Platform that ingests structured billing data from a CSV file, performs multi-dimensional cost analysis using Python and Pandas, generates visual insights via Matplotlib, and exports comprehensive Excel reports using OpenPyXL — all within a Jupyter Notebook environment requiring no external API dependencies.

Technologies

The platform is built entirely using open-source Python libraries:

- Python 3.x — Core execution environment and scripting language
- Pandas (≥ 1.3) — Data ingestion, groupby aggregation, pivot table construction, and reindexing
- Matplotlib (≥ 3.4) — Generation of pie charts and bar charts for visual cost analysis
- OpenPyXL (≥ 3.0) — Multi-sheet Excel workbook creation for stakeholder reporting
- Jupyter Notebook — Interactive development and execution environment

Hardware Architecture

The platform is designed for execution on any standard development machine. No specialised hardware, GPU, or cloud infrastructure is required. The pipeline processes 60 billing records in under five seconds on a typical laptop or desktop computer with at least 4 GB RAM. All input and output artefacts are managed through the local filesystem.

Software Architecture

The platform is structured as a sequential, four-tier data processing pipeline: an Ingestion Tier (CSV Loader via Pandas), a Processing Tier (Analytics Engine for grouping and aggregation), a Visualisation Tier (Chart Generator via Matplotlib), and an Export Tier (Report Writer via OpenPyXL). Each tier is independently testable and replaceable without disrupting the core analytical logic.

System

Requirements

Functional Requirements

The Cloud Cost Intelligence Platform must fulfil the following functional requirements:

- FR-01: Ingest a structured CSV billing file (smart_city_billing.csv) containing 60 records across 6 columns and load it into a Pandas DataFrame with full schema validation.
- FR-02: Perform exploratory data analysis including record count, column metadata, basic statistics, and null-value detection, logging results to the console.
- FR-03: Aggregate total cloud expenditure by Service Type (Compute, Storage, Networking, Database, AI/ML Services) and rank services by cost in descending order.
- FR-04: Aggregate total cloud expenditure by Month across all twelve calendar months, reindexed in chronological order to support temporal trend analysis.
- FR-05: Aggregate total cloud expenditure by Department (Traffic Management, Public Safety, Smart Lighting, Waste Management, Water Supply) in ascending order.
- FR-06: Construct a Month × Service Type pivot table to enable multi-dimensional cost analysis across time and service categories.
- FR-07: Generate four chart visualisations — an exploded pie chart by service type, a pie chart with dollar labels by month, a stacked bar chart of monthly spend by service, and a horizontal bar chart by department — and save each as a PNG file at 120 DPI.
- FR-08: Export all aggregated summaries into a structured, multi-sheet Excel workbook (cloud_cost_report.xlsx) with separate sheets for Service Summary, Monthly Summary, and Department Summary.

User Requirements

The platform is designed to serve the following user groups and their associated requirements:

- Smart City Administrators: Require a clear, visual summary of cloud expenditure across departments and months to support quarterly budget reviews and infrastructure planning decisions.
- Data Analysts: Require access to structured aggregated data in Excel format for further statistical analysis, trend modelling, or integration with business intelligence tools.
- Academic Evaluators: Require a reproducible, self-contained Jupyter Notebook that demonstrates applied data engineering, visualisation, and reporting competencies against the project brief PRJN26-108.
- IT Managers: Require actionable insights into which cloud service categories are consuming the highest share of the city's cloud budget to guide procurement and optimisation decisions.

Environmental Requirements

The platform operates within the following environmental constraints:

- Python 3.x must be installed with the following packages available: pandas (≥ 1.3), matplotlib (≥ 3.4), openpyxl (≥ 3.0), and warnings (standard library).
- The input file smart_city_billing.csv must be present in the same working directory as the Jupyter Notebook before execution begins.
- The execution environment must have read and write permissions on the working directory to load the input CSV and save output PNG and XLSX artefacts.
- Jupyter Notebook or JupyterLab must be installed and running for interactive cell-by-cell execution. The notebook may also be executed as a script via jupyter nbconvert --to script.
- No internet connectivity, external API keys, or cloud service accounts are required at any stage of the pipeline.

Design and Architecture

The platform follows a clean separation of concerns across four tiers, each with well-defined inputs and outputs. The architecture is designed for extensibility, reproducibility, and academic transparency.

Application Architecture Overview

Tier	Component	Responsibility
Ingestion Tier	CSV Loader (Pandas)	Reads and validates smart_city_billing.csv into a structured DataFrame
Processing Tier	Analytics Engine (Pandas)	Performs grouping, aggregation, pivot operations, and rolling statistics
Visualisation Tier	Chart Generator (Matplotlib)	Produces pie charts and bar charts for service-type, monthly, and departmental views
Export Tier	Report Writer (OpenPyXL)	Serialises summary tables into a formatted multi-sheet Excel workbook

Data Model

The system's persistent input model relies on a single flat CSV file. Each row represents one cloud service invoice record for a specific department in a specific month.

Field	Type	Description
Invoice_ID	String	Unique invoice identifier (e.g., INV-001)
Month	String	Calendar month name (January through December)
Service_Type	String	Compute, Storage, Networking, Database, or AI/ML Services
Department	String	Traffic Mgmt, Public Safety, Smart Lighting, Waste Mgmt, or Water Supply
Usage_Hours	Integer	Total hours the cloud service was consumed during the billing period
Cost_USD	Float	Total monetary cost in US Dollars for the billing record

Process Flow

The high-level operational flow executes these sequential steps within the Jupyter Notebook:

- Step 1 — Data Ingestion: The billing CSV is loaded into a Pandas DataFrame. Column types are validated and global display settings are configured.
- Step 2 — Exploratory Analysis: Dataset dimensions, column metadata, basic statistics, and null-value counts are computed and logged.
- Step 3 — Service-Type Aggregation: Records are grouped by Service_Type, Cost_USD is summed, and results are sorted descending.
- Step 4 — Monthly Aggregation: Records are grouped by Month, reindexed to calendar order, and summed for temporal trend analysis.
- Step 5 — Departmental Aggregation: Records are grouped by Department, revealing which city operations consume the most cloud budget.
- Step 6 — Pivot Analysis: A Month × Service_Type pivot table is constructed for stacked bar visualisation.
- Step 7 — Visualisation: Four pie and bar charts are rendered and saved as PNG artefacts at 120 DPI.
- Step 8 — Excel Export: All aggregated summaries are written into a structured, multi-sheet Excel workbook.

Technology Stack

Library	Version	Purpose
Python	3.x	Core execution environment
Pandas	≥ 1.3	Data ingestion, grouping, aggregation, and pivot operations
Matplotlib	≥ 3.4	Pie chart and bar chart generation
OpenPyXL	≥ 3.0	Multi-sheet Excel report export
Warnings	stdlib	Suppression of non-critical runtime warnings
Jupyter Notebook	Any	Interactive development and execution environment

Implementation

The Cloud Cost Intelligence Platform is implemented as a sequential Jupyter Notebook. Each cell builds upon the results of the previous cell. The pipeline is entirely self-contained, requiring only the input CSV file and standard Python libraries.

Module 1 — Data Ingestion

The ingestion module performs environment setup and data loading. All required libraries (pandas, matplotlib, openpyxl, warnings) are imported, float display precision is set to two decimal places, chart DPI is configured to 100 for screen rendering, and warnings are suppressed. The CSV is loaded via pd.read_csv() into a master DataFrame df with expected shape (60, 6). The module validates load success by printing record count, column metadata, and the first ten rows.

Module 2 — Aggregation Engine

Three independent groupby aggregations are computed from the master DataFrame:

- Service-Type Aggregation: df.groupby('Service_Type')['Cost_USD'].sum().sort_values(ascending=False) — groups all 60 records by service category and ranks by total expenditure.
- Monthly Aggregation: df.groupby('Month')['Cost_USD'].sum().reindex(month_order) — groups records by month and reindexes to calendar order to preserve chronological integrity.
- Departmental Aggregation: df.groupby('Department')['Cost_USD'].sum().sort_values(ascending=True) — groups by department, sorted ascending so the horizontal bar chart reads naturally.

Module 3 — Pivot and Temporal Analysis

A 12 × 5 pivot table is constructed using df.pivot_table(values='Cost_USD', index='Month', columns='Service_Type', aggfunc='sum'), then reindexed to calendar order. This structure provides the direct input to the stacked bar chart and enables multi-dimensional spend analysis combining both month and service category dimensions in a single DataFrame operation.

Module 4 — Visualisation

Four distinct chart visualisations are produced and saved as PNG files at 120 DPI:

- Service Pie Chart (pie_by_service.png): An exploded pie chart showing the percentage and dollar share of total expenditure per cloud service type.

- Monthly Pie Chart (pie_by_month.png): A pie chart with custom lambda autopct labels displaying both dollar value and percentage share for each month.
- Stacked Bar Chart (bar_monthly_stacked.png): A month-by-month stacked bar chart breaking down spend by service type, with per-bar total annotations.
- Department Bar Chart (bar_by_department.png): A horizontal bar chart displaying total expenditure per smart city department, with value labels and 20% x-axis padding.

All charts are saved via plt.savefig(filename, bbox_inches='tight', dpi=120) before plt.show() to prevent legend clipping.

Module 5 — Excel Export

The export module uses pd.ExcelWriter with the openpyxl engine as a context manager to write three summary sheets — Service Summary, Monthly Summary, and Department Summary — into a single workbook (cloud_cost_report.xlsx). Each aggregated Series is converted to a DataFrame before writing, preserving the groupby key as a labelled index column. The context manager ensures the file handle is properly flushed and closed even in the event of an exception.

Testing

Graphical User Interface (GUI) Layout

The platform's primary interface is the Jupyter Notebook environment, which provides an interactive cell-by-cell execution model. The notebook is structured into clearly labelled sections corresponding to each pipeline stage. Inline chart rendering within the notebook provides immediate visual feedback after each visualisation cell executes. Console log outputs (print statements) serve as progress indicators confirming successful completion of each analytical step.

The four output charts (pie_by_service.png, pie_by_month.png, bar_monthly_stacked.png, bar_by_department.png) and the Excel workbook (cloud_cost_report.xlsx) are written to the working directory and can be accessed directly from the filesystem or opened within the notebook via display utilities.

Customer Testing

The platform was tested against the following scenarios to validate functional correctness and output quality:

Test Case	Input	Expected Output	Result
TC-01: CSV Load	smart_city_billing.csv (60 records)	DataFrame shape (60, 6), no null values	Pass
TC-02: Service Aggregation	df grouped by Service_Type	5-element Series sorted descending	Pass
TC-03: Month Ordering	df grouped by Month	12-element Series in Jan–Dec order	Pass
TC-04: Dept. Aggregation	df grouped by Department	5-element Series sorted ascending	Pass
TC-05: Pivot Table	df pivot on Month × Service_Type	12×5 DataFrame, calendar-ordered rows	Pass

TC-06: Chart Export	service_cost, monthly_cost, pivot, dept_cost	4 PNG files at 120 DPI, no clipping	Pass
TC-07: Excel Export	service_cost, monthly_cost, dept_cost	3-sheet XLSX with labelled columns	Pass
TC-08: Month Reindex Guard	Alphabetically-sorted month groupby	Calendar-correct order after reindex	Pass

Evaluation

The platform successfully met all eight functional requirements defined in the System Requirements section. The calendar-correct month reindexing mechanism correctly resolves Python's default alphabetical groupby ordering, which would otherwise render temporal charts meaningless. The closure-based lambda autopct on the monthly pie chart accurately converts percentage arguments back to absolute dollar values for human-readable labels. The shared master DataFrame design eliminates redundant file I/O across all downstream analytical stages, and the context-manager-protected Excel export ensures file integrity under all execution conditions.

Performance benchmarking confirmed that the complete pipeline executes in under five seconds on a standard development machine for the 60-record dataset. Pandas' vectorised operations ensure that this performance scales linearly with dataset size without requiring algorithmic restructuring for datasets up to several hundred thousand records.

Snapshots of the Project

The following section describes the key output artefacts generated by the Cloud Cost Intelligence Platform. All charts are saved as high-resolution PNG files (120 DPI) to the project working directory.

Output Artefact Summary

Artefact	Format	Filename	Contents
Service Pie Chart	PNG (120 DPI)	pie_by_service.png	Exploded pie showing % and $ share per service type
Monthly Pie Chart	PNG (120 DPI)	pie_by_month.png	Pie showing monthly $ cost and % share for Jan–Dec
Stacked Bar Chart	PNG (120 DPI)	bar_monthly_stacked.png	Stacked bar of monthly spend broken down by service type
Dept. Bar Chart	PNG (120 DPI)	bar_by_department.png	Horizontal bar of total spend per smart city department
Excel Report	XLSX	cloud_cost_report.xlsx	3-sheet workbook: Service Summary, Monthly Summary, Dept Summary

The Service Pie Chart provides an at-a-glance breakdown of where the majority of cloud budget is allocated across the five service categories. The Monthly Pie Chart reveals seasonal or operational spending patterns across the twelve-month period. The Stacked Bar Chart provides the richest multi-dimensional view, displaying how each service category contributes to month-by-month expenditure fluctuations. The Department Horizontal Bar Chart enables city administrators to directly compare operational cloud costs across all five smart city departments.

Conclusions

The Cloud Cost Intelligence Platform for Smart City successfully demonstrates the application of Python-based data engineering and analytical skills to a real-world IT management problem — cloud cost optimisation for municipal infrastructure. The project achieved all stated functional objectives: structured data ingestion, multi-dimensional cost aggregation, calendar-correct temporal analysis, multi-format visualisation, and structured Excel reporting.

The platform's modular architecture — separating ingestion, processing, visualisation, and export into independent tiers — ensures that each component can be modified, tested, or replaced without disrupting the analytical core. The explicit month reindexing mechanism, the closure-based autopct lambda, and the context-manager-protected Excel export collectively represent production-quality engineering decisions applied within an academic context.

This project highlights that meaningful cloud cost intelligence can be delivered through a lightweight, dependency-minimal Python pipeline — without requiring enterprise FinOps platforms, cloud-native tooling, or external API integrations. The same analytical architecture is directly transferable to real smart city deployments with expanded datasets spanning multiple years, cloud providers, and geographic regions.

Further Development or Research

Several directions exist for extending this platform beyond its current academic scope:

- Interactive Dashboard: The Pandas analytics layer can be wrapped in Streamlit or Plotly Dash to deliver a browser-based, interactive cost exploration interface for non-technical city administrators.
- Multi-Provider Support: The CSV schema can be extended to include a Cloud_Provider column (AWS, Azure, GCP), enabling comparative cross-provider cost analysis and vendor optimisation recommendations.
- Anomaly Detection: A rolling statistics layer (3-month or 6-month moving averages) can be added to automatically flag months or departments where expenditure exceeds expected thresholds, enabling proactive cost control.
- Forecasting Module: Time-series forecasting using Facebook Prophet or statsmodels ARIMA can project future cloud expenditure based on historical billing patterns, supporting annual budget planning.
- Database Integration: The in-memory DataFrame model can be extended to persist billing records to a PostgreSQL or SQLite database, enabling historical trend analysis across multiple fiscal years.
- Automated Report Delivery: The Excel export module can be augmented with email automation (smtplib) or cloud storage integration (AWS S3, Azure Blob) to distribute reports to department heads on a scheduled basis.

References

[1] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

[2] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

[3] OpenPyXL Documentation: https://openpyxl.readthedocs.io — Excel file read/write operations in Python.

[4] Pandas Documentation: https://pandas.pydata.org/docs — DataFrame groupby, pivot_table, and reindex API references.

[5] Jupyter Project: https://jupyter.org/documentation — Interactive computing and notebook execution environment.

[6] Smart City Cloud Billing Dataset: smart_city_billing.csv — Internal project dataset covering 5 service types, 5 departments, and 12 months of cloud expenditure records (PRJN26-108).

[7] High Level Design (HLD) Document: Cloud Cost Intelligence Platform for Smart City — PRJN26-108, Yenepoya University, 2025–26.

[8] Low Level Design (LLD) Document: Cloud Cost Intelligence Platform for Smart City — PRJN26-108, Yenepoya University, 2025–26.

Appendix

Appendix A — Input CSV Schema

The input file smart_city_billing.csv must conform to the following schema. Any deviation will raise a KeyError during groupby operations.

Column Name	Dtype	Constraints	Example Value
Invoice_ID	str	Non-null, unique	INV-001
Month	str	Full month name (January…December)	January
Service_Type	str	Compute, Storage, Networking, Database, AI/ML Services	Compute
Department	str	Traffic Management, Public Safety, Smart Lighting, Waste Management, Water Supply	Traffic Management
Usage_Hours	int	Non-null, ≥ 0	720
Cost_USD	float	Non-null, ≥ 0.0	1440.00

Appendix B — Configuration Constants

Constant / Setting	Value	Purpose
plt.rcParams['figure.dpi']	100	Screen rendering resolution for notebook display
savefig dpi	120	Higher resolution for saved PNG chart artefacts
display.float_format	'{:.2f}'.format	Two-decimal precision for DataFrame print output
month_order	List of 12 month name strings	Canonical calendar ordering for reindex operations
Service colour palette	#2196F3, #4CAF50, #FF9800, #E91E63, #9C27B0	5-colour mapping for service types across all charts

Excel engine	openpyxl	Backend engine for pd.ExcelWriter (.xlsx format)

Appendix C — Error Handling Reference

Error Condition	Pandas Behaviour	Recommended Guard
CSV file not found	FileNotFoundError at pd.read_csv()	Wrap in try/except FileNotFoundError with descriptive message
Missing required column	KeyError at first groupby call	Validate df.columns contains all 6 expected column names after load
Non-numeric Cost_USD	TypeError at .sum()	Apply pd.to_numeric(df['Cost_USD'], errors='coerce') and check NaN
Unknown month name	NaN row after reindex()	Assert set(df['Month'].unique()).issubset(set(month_order))
Empty DataFrame	ValueError on zero-sum pie chart	Check len(df) > 0 immediately after load